

feature_engineering.py

Every feature-set's preprocessing pipeline, plus the two learned encoders (GBDT-leaf, FM)

This is the project's core module: it defines the sklearn preprocessing pipelines for all seven feature sets the project compares (see `trainer.py`), the smoothed/leave-one-out target-encoding math behind `freq_agg`, and two custom sklearn-compatible transformers — `GBDTLeafEncoder` and `FMEEmbeddingEncoder` — that each turn a fitted model's internal structure into new induced features for a downstream linear model.

build_preprocessing_pipeline **FUNCTION**

```
build_preprocessing_pipeline(cat_cols, num_cols, max_categories=50) -> ColumnTransformer
```

The preprocessor for the `freq_agg` feature set. One-hot encodes `cat_cols` (with unseen categories at transform time mapped to an all-zero row via `handle_unknown="ignore"`, and the `max_categories` cap bucketing anything beyond the 50 most frequent values per column into a single "infrequent" catch-all — this keeps the low-cardinality context columns' one-hot block manageable), and standardizes `num_cols` (the smoothed target-encoded `_ctr / _count` columns plus `hour_of_day / day_of_week`) to zero mean, unit variance.

Output is dense (`sparse_output=False`), required because `HistGradientBoostingClassifier` — one of the models this feature set can pair with — needs dense input.

Standardization of each numeric column

$$Z = \frac{x - \mu}{\sigma}$$

build_ohe_only_pipeline **FUNCTION**

```
build_ohe_only_pipeline(cat_cols) -> ColumnTransformer
```

The preprocessor for the `baseline_ohe` feature set: one-hot encodes every raw categorical column (including high-cardinality `device_id / device_ip / site_id / app_id`) with **no** `max_categories` cap and no numeric branch — a deliberately naive baseline with no target-encoding at all.

Output is sparse (`sparse_threshold=1.0` forces sparse regardless of density heuristics) — required for this to be tractable, since uncapped one-hot over these columns produces on the order of 500K–600K columns on the project's default 2M-row sample. Only sparse-input-capable models (`LogisticRegression` , `lightgbm`) can pair with this feature set.

to_lgbm_categoricals **FUNCTION**

```
to_lgbm_categoricals(train_df, val_df, cat_cols) -> (train_df, val_df)
```

Casts `cat_cols` to pandas' `category` dtype so `lightgbm` can use its native categorical-split handling (rather than one-hot) for the `gbdt_leaves` feature set. Each column's category *vocabulary* is fixed from `train_df` only — `val_df`'s values are cast against that same fixed vocabulary, so any value never seen in training becomes `NaN` (which `lightgbm` treats as a "missing" branch at split time, not an error). This is the no-leakage rule applied to dtype casting: the category vocabulary is exactly the kind of train-derived state that must never be re-fit on validation data.

GBDTLeafEncoder (class) CLASS

```
GBDTLeafEncoder(BaseEstimator, TransformerMixin)
```

Implements the Facebook 2014 ads-CTR "GBDT + LR" technique: fit a gradient-boosted decision tree ensemble, then re-encode every row not by its raw features but by *which leaf of which tree it landed in*.

`fit(X, y)` trains a `lightgbm.LGBMClassifier` (hyperparameters from `gbdt_params`) on `(X, y)`, then calls `gbdt_.predict(X, pred_leaf=True)`, which returns — instead of a prediction — an $(n_samples, n_estimators)$ integer matrix: for tree `t` and row `i`, the index of the leaf that row falls into. A fresh `OneHotEncoder` is then fit on that integer matrix, treating each tree's column as its own categorical variable.

`transform(X)` re-applies the already-fit GBDT and one-hot encoder to new data (train or otherwise), returning the one-hot leaf matrix.

Why this is a useful feature representation: each tree partitions the input space into a small number of leaves via a sequence of learned, non-linear, interaction-aware splits. Whether row i shares a leaf with row j in tree t is a strong, highly non-linear similarity signal that a subsequent *linear* model (logistic regression) cannot discover on its own from the raw columns — the GBDT does the non-linear feature engineering, and the linear model just has to learn one weight per (tree, leaf) combination.

Induced feature representation, T trees with leaf-index functions leaf_t

$$\phi(x) = \bigoplus_{t=1}^T \text{onehot}(\text{leaf}_t(x)), \quad \text{leaf}_t(x) \in \{1, \dots, L_t\}$$

build_gbd_t_leaves_ohe_pipeline FUNCTION

```
build_gbd_t_leaves_ohe_pipeline(cat_cols, gbd_t_params=None) -> Pipeline
```

The preprocessor for the `gbd_t_leaves_ohe` feature set: one-hot encode `cat_cols` (via `build_ohe_only_pipeline`), then feed that sparse matrix into `GBDTLeafEncoder`. Training the GBDT on already-one-hot input (rather than raw categoricals with `lightgbm`'s native handling, used by the `gbd_t_leaves` feature set) avoids `lightgbm`'s `max_bin` cap silently losing resolution on the highest-cardinality columns.

build_gbd_t_leaves_concat_pipeline FUNCTION

```
build_gbd_t_leaves_concat_pipeline(cat_cols, gbd_t_params=None) -> FeatureUnion
```

The preprocessor for the `gbd_t_leaves_concat` feature set: concatenates (via `sklearn.pipeline.FeatureUnion`) the raw uncapped one-hot vectors with the GBDT-leaf one-hot features from `build_gbd_t_leaves_ohe_pipeline`, so the downstream linear model sees *both* the original linear signal and the GBDT's induced non-linear signal — matching the original Facebook paper's actual design, where leaves *augment* the base features rather than replacing them (unlike the leaf-only `gbd_t_leaves_ohe`). Both branches independently fit their own `OneHotEncoder` on the same columns; `FeatureUnion` does not share fitted state across branches, a small duplicated cost cheap relative to the GBDT/LR fits.

build_freq_agg_leaves_concat_pipeline FUNCTION

```
build_freq_agg_leaves_concat_pipeline(freq_cat_cols, freq_num_cols, gbd_t_cat_cols,
gbd_t_params=None) -> FeatureUnion
```

The preprocessor for the `freq_agg_leaves_concat` feature set: concatenates `freq_agg`'s engineered features with GBDT-leaf one-hot features — but the internal GBDT is deliberately trained on raw uncapped one-hot (`gbd_t_cat_cols`, the same columns `gbd_t_leaves_ohe` uses), **not** on `freq_agg`'s own `_ctr` / `_count` columns.

This is a load-bearing design decision, not an arbitrary choice. `_ctr` is computed via **leave-one-out** for training rows (see `add_freq_agg_features` below): each row's own label is subtracted from its own category's statistics before computing that row's smoothed CTR. This introduces a small, systematic *train-only* artifact — a clicked row's own `_ctr` value is nudged slightly lower than an otherwise-identical unclicked row's, purely because its own click was excluded from the numerator. A linear model barely notices this (swamped by the real cross-category signal), but a second, more flexible GBDT trained directly on those columns *can and empirically does* exploit it: in testing, a GBDT trained on `freq_agg`'s dense matrix scored $AUC \approx 0.48$ (worse than random) on held-out validation, because the leaf splits it learned encoded that train-only artifact rather than genuinely transferable signal. Training the leaf-generating GBDT on plain one-hot instead avoids this failure mode entirely, since raw category indicators carry no target-derived, self-referential information.

FMEEmbeddingEncoder (class) CLASS

```
FMEEmbeddingEncoder(BaseEstimator, TransformerMixin)
```

A from-scratch, dependency-free implementation of a degree-2 Factorization Machine (Rendle, "Factorization Machines", 2010), trained via minibatch SGD directly on sparse one-hot input. No external FM library (pyfm/fastFM/xlearn) is used — those are largely unmaintained C-extension packages with a real risk of build failure on modern Python/ARM; this implementation uses only numpy/scipy, both already project dependencies.

The model. A standard degree-2 FM predicts a real-valued score for input vector x as a global bias, plus a per-feature linear term, plus a pairwise interaction term where every pair of features i, j gets its own learned "compatibility" via the dot product of two k -dimensional latent vectors v_i, v_j (one per feature, shared across all pairs that feature participates in — this is what makes FMs tractable even when most pairwise feature combinations are never observed together in the training data, unlike a naive full pairwise-interaction model):

Computing this pairwise sum naively costs $O(d^2)$ per row. The standard FM trick re-expresses it in $O(dk)$:

For this project's specific case — x is a one-hot indicator vector, so $x_i \in \{0, 1\}$ and therefore $x_i^2 = x_i$ — the second inner sum simplifies from $\sum_i v_i f^2 x_i^2$ to $\sum_i v_i f^2 x_i$, which is exactly what the code computes as `sq_sum = Xb_active @ (v_active**2)` (a sparse matrix product, no explicit squaring of x needed). The corresponding line `s = Xb_active @ v_active` computes $s_f = \sum_i v_i f x_i$ — for a one-hot row, this is literally the sum of the latent vectors of that row's active categories, i.e. an embedding of the row. This `s` vector is exactly what `transform()` returns as the induced feature: the FM's own internal per-row representation, exposed for a downstream linear model to use, the FM analog of `GBDTLeafEncoder`'s leaf one-hot vectors.

Training objective. The FM's raw score $\hat{y}(x)$ is passed through a logistic sigmoid to get a click probability, and trained by minimizing mean binary cross-entropy:

Gradients. Writing `err = p - y` (the standard logistic-loss gradient with respect to the pre-sigmoid score), the three parameter gradients are:

The v gradient again exploits $x_i^2 = x_i$: the raw per-example gradient `$\partial \hat{y} / \partial v_i f = x_i (s_f - v_i f \cdot x_i)$` only needs s (already computed for the forward pass) and the model's own parameters — no second matrix multiply against a materialized pairwise-interaction tensor is ever required. In code, the batch-vectorized form is `grad_v_active = (Xb_active.T @ (err[:, None] * s)) / nb - v_active * (g_active / nb)[:, None]`.

SGD updates with L2 regularization (learning rate η , regularization strength $\lambda = \text{`l2\_reg`}$):

Sparsity-restricted updates. A typical row activates only `len(cat_cols)` (roughly 20) of up to ~600K one-hot columns. Naively updating the full (d, k) parameter matrix v every minibatch would cost $O(dk)$ regardless of batch sparsity. Instead, each step restricts every dense computation to `active_cols =`

`np.unique(Xb.indices)` — the columns actually touched by that batch — via `Xb_active = Xb[:, active_cols]`, `v_active = self.v_[active_cols]`, etc., and writes updates back only to those rows with `self.v_[active_cols] -= ...`. This makes each step's cost proportional to the batch's actual sparsity, not d .

Initialization. w and w_0 start at zero, but v must be initialized with small random values ($\mathcal{N}(0, 0.01^2)$), not zero: with $v = 0$, the projection $s = Xv$ is identically 0, which — inspecting the gradient formula above — makes $\partial \hat{y} / \partial v_{i,f}$ identically 0 too, a genuine degenerate fixed point that zero-initialization could never escape from under gradient descent.

Early stopping. `fit` tracks the best (sample-weighted, across variable-sized batches) training loss seen so far each epoch; if no epoch improves on that best by more than `early_stopping_tol` for `early_stopping_patience` consecutive epochs, training stops early. This makes `n_epochs` an upper bound rather than a value that must be precisely retuned — raising it to explore whether more training helps is safe rather than risking silently wasted compute or overfitting on an unattended run.

Degree-2 FM prediction (raw score, before sigmoid)

$$\hat{y}(x) = w_0 + \sum_{i=1}^d w_i x_i + \sum_{i=1}^d \sum_{j=i+1}^d \langle v_i, v_j \rangle x_i x_j$$

$O(dk)$ reformulation of the pairwise term (the standard FM trick)

$$\sum_{i < j} \langle v_i, v_j \rangle x_i x_j = \frac{1}{2} \sum_{f=1}^k \left[\left(\sum_{i=1}^d v_{i,f} x_i \right)^2 - \sum_{i=1}^d v_{i,f}^2 x_i^2 \right]$$

Sigmoid link and binary cross-entropy loss over a batch of size n

$$p = \sigma(\hat{y}) = \frac{1}{1 + e^{-\hat{y}}}, \quad L = -\frac{1}{n} \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

Parameter gradients (s_f = sum of active latent vectors' f -th coordinate)

$$\frac{\partial L}{\partial w_0} = \overline{\text{err}}, \quad \frac{\partial L}{\partial w_i} = \overline{\text{err} \cdot x_i}, \quad \frac{\partial L}{\partial v_{i,f}} = \overline{\text{err} \cdot x_i (s_f - v_{i,f} x_i)}$$

$$\theta \leftarrow \theta - \eta(\nabla_{\theta}L + \lambda\theta)$$

build_fm_embed_pipeline **FUNCTION**

```
build_fm_embed_pipeline(cat_cols, fm_params=None) -> Pipeline
```

One-hot encode `cat_cols` (via `build_ohe_only_pipeline`), then feed into `FMEEmbeddingEncoder` — the FM analog of `build_gbd_t_leaves_ohe_pipeline`, reused by `build_freq_agg_fm_concat_pipeline` below.

build_freq_agg_fm_concat_pipeline **FUNCTION**

```
build_freq_agg_fm_concat_pipeline(freq_cat_cols, freq_num_cols, fm_cat_cols,
fm_params=None) -> FeatureUnion
```

The preprocessor for the `freq_agg_fm_concat` feature set: concatenates `freq_agg`'s engineered features with `FMEEmbeddingEncoder`'s latent projection features. Mirrors `build_freq_agg_leaves_concat_pipeline`'s design for the identical reason: the FM is trained on raw uncapped one-hot (`fm_cat_cols`), not on `freq_agg`'s own leave-one-out-adjusted `_ctr / _count` columns, since those carry the same train-only artifact a flexible second model could otherwise exploit.

add_hour_features **FUNCTION**

```
add_hour_features(df, hour_col=HOUR_COL) -> pd.DataFrame
```

Parses Avazu's `hour` column — an integer/string in `YYMMDDHH` format, e.g. `14102100` = 2014-10-21, hour 00 — into two derived columns: `hour_of_day` (the trailing two digits, 0–23, parsed directly as a substring) and `day_of_week` (via constructing a full date from the `YY / MM / DD` substrings — with century assumed to be 2000 + YY — and taking pandas' `.dt.dayofweek`, Monday=0).

fit_freq_agg_stats **FUNCTION**

```
fit_freq_agg_stats(train_df, group_cols, label_col=LABEL_COL) -> dict[str, pd.DataFrame]
```

For each column in `group_cols` (the high-cardinality user/ad columns), groups `train_df` by that column and aggregates the label into a per-category `(sum, count)` pair — total clicks and total impressions for that category value, computed on the training split only. Raw sums/counts are kept (rather than a precomputed mean) specifically so the smoothing strength can be chosen later, at apply time, without re-aggregating.

apply_freq_agg_stats **FUNCTION**

```
apply_freq_agg_stats(df, fit_stats, global_ctr, smoothing=DEFAULT_SMOOTHING) ->  
pd.DataFrame
```

Applies previously-fit per-category stats to any DataFrame (used for the validation split, and for scoring new data at inference time) as a smoothed, Bayesian-style CTR estimate: for a category with `s` total clicks over `N` impressions in training, blended toward the global click rate $\bar{p}$ by `m = smoothing` "pseudo-observations":

This is equivalent to pretending `m` extra impressions at exactly the global rate were observed before any real data for that category — a category seen only a handful of times gets pulled heavily toward $\bar{p}$ (protecting against noisy small-sample estimates), while a category seen tens of thousands of times is barely nudged. A category *never* seen in training has `S=N=0`, which the formula resolves to exactly $\bar{p}$ — a clean, principled fallback with no special case needed. `DEFAULT_SMOOTHING = 20.0`.

Smoothed (additive) target encoding

$$\text{ctr}_{\text{smooth}} = \frac{S + m\bar{p}}{N + m}$$

add_freq_agg_features FUNCTION

```
add_freq_agg_features(train_df, val_df, group_cols, label_col=LABEL_COL,
                      smoothing=DEFAULT_SMOOTHING) -> (train_df, val_df, fit_stats)
```

Orchestrates the full leakage-safe target-encoding procedure for every column in `group_cols`, adding a `{col}_ctr` and `{col}_count` column to both DataFrames — but train and validation are handled by two genuinely different mechanisms, not the same formula applied twice.

`val_df` gets the fitted stats applied exactly as in `apply_freq_agg_stats` above — safe, because no validation row contributed to those statistics in the first place.

`train_df` cannot use the same fitted stats directly: doing so would let each row's feature value be computed *using its own label*, a direct leakage channel. Instead each row gets a **leave-one-out (LOO)**, **smoothed** estimate — its own label is first subtracted out of its category's sum/count, *then* the same additive-smoothing formula is applied to the remainder:

Plain leave-one-out with no smoothing is not sufficient on its own: for a category group of size 2, one row's leave-one-out estimate reduces to exactly the *other* row's raw label — a near-literal leak that a sufficiently flexible model can still exploit to memorize training rows via high-cardinality columns. This was confirmed empirically during development: unsmoothed LOO gave a GBDT trained downstream a train AUC of 1.0 but a validation AUC of only ≈ 0.51 — pure overfitting to near-unique small groups. Blending toward `global_ctr` by `smoothing` pseudo-observations (identically to the val-side formula) bounds how much any single group, however small, can shift its members' feature value, closing that hole.

Returns `fit_stats` — the explicit fit-state from `fit_freq_agg_stats` — alongside the two engineered DataFrames, so it can be persisted with a trained model (see `trainer.save_model`) and reapplied to score fresh raw data later without needing the original training set.

Leave-one-out smoothed target encoding for a training row with its own label y_i in a group of total sum S_g , count N_g

$$\text{ctr}_i^{\text{train}} = \frac{(S_g - y_i) + m \bar{p}}{(N_g - 1) + m}$$